Semestre 6 Année universitaire 2025 - 2026

Rapport - Projet POO-IHM2

licence 3 informatique

Auteurs

Lucas [redacted]
Mathis
Tom
Léonard

Université de Poitiers

FACULTÉ DES SCIENCES FONDAMENTALES ET APPLIQUÉES UNIVERSITÉ DE POITIERS

Sommaire

I - Contexte et objectifs.....3
II - Choix de conception.....5
III - Difficultés rencontrées.....7
IV - Les principes de la POO dans notre projet.....8
V - Gestion de projet.....12
VI - Conclusion.....14

I - Contexte et objectifs

INTRODUCTION :

Ce projet s'inscrit dans le cadre de l'enseignement de POO-IHM2. Il a pour objectif de nous faire mettre en pratique les notions de programmation orienté objet, de conception logicielle ainsi que de développement d'interface homme-machine. Le tout en respectant le modèle MVC (Model-Vue-Contrôleur).

Le sujet repose sur la réalisation de deux applications complémentaires :

- une interface graphique pour un jeu d'aventure 2D.
- un éditeur de niveaux, permettant de créer et jouer sur des cartes personnalisées.

Au-delà de l'aspect purement technique, ce projet constitue également un exercice pour ce qui est de la conduite de projet. Il nous a amené à organiser notre travail en équipe, à prendre des choix de conceptions importants, à nous répartir les tâches et les responsabilités, mais aussi à contrôler l'évolution du code à l'aide d'un système de versionnement. Nous avons dû produire plusieurs livrables avec notamment un rendu intermédiaire, une documentation technique, un manuel utilisateur ainsi qu'un rapport final.

CONTEXTE :

Le projet se base sur un code existant : le développement d'un jeu d'aventure textuel que nous avons réalisé au S5, autour d'un jeu d'exploration. Il se base donc sur du code existant, à partir duquel nous repartirons pour développer la suite.

Effectivement, le cahier des charges demande une évolution vers une application plus complète, structurée selon le modèle de conception MVC vu en cours. L'un des enjeux principaux est donc de bien organiser notre code en suivant ce modèle. Il sera donc divisé en trois parties : le Modèle, la Vue et le Contrôleur, ce qui permettra de proposer plusieurs interfaces différentes, sans remettre en cause la logique métier.
Cette exigence justifie la majorité des choix pour l'architecture réalisée.

EQUIPE :

Pour réaliser ce projet, notre groupe était composé de :

- Lucas Vadé
- Mathis Krizmanic
- Tom Charpentier
- Léonard Duhem

Le travail a été mené de manière collective, avec une répartition progressive des responsabilités selon les compétences, les disponibilité de chacun et l'avancement global du projet

OBJECTIFS DU PROJET :

L'objectif principal du projet était de concevoir une application logicielle complète respectant le modèle MVC, afin de mettre en œuvre les concepts étudiés en POO-IHM. Il s'agissait non seulement de produire une application fonctionnelle mais aussi de démontrer les points forts d'une architecture qui sépare les responsabilités entre la logique métier, le traitement et l'interaction utilisateur.

Ce projet regroupe donc plusieurs objectifs dont voici les plus importants :

- Adapter notre projet du Semestre 5 au format MVC

- Développer une nouvelle interface graphique avec JavaFX, complémentaire à la version du jeu textuel déjà jouable. De nouvelles mécaniques apparaîtront également : un mode multijoueur, des paramètres de configuration, un système de scores ou encore une option de changement de langue.

- La réalisation d'un éditeur de niveau permettant de créer, charger et sauvegarder des cartes. Il se présente sous la forme d'une grille dans laquelle différents éléments du jeu peuvent être placés.

II - Choix de conception

1. Adaptation du projet au modèle MVC

Si l'un des objectifs de ce projet était l'ajout d'une nouvelle interface graphique, un autre point majeur à prendre en compte fut l'adaptation de notre jeu au format MVC (Model-View-Controller), conformément à l'architecture imposée dans le cadre du projet.

Cette contrainte imposait donc une séparation stricte entre les différents "rôles" du programme. Par exemple, l'un des points à prendre en compte avec le modèle MVC, est que le Modèle et la Vue ne communiquent jamais directement entre eux. Toute interaction doit passer par le Contrôleur, qui joue le rôle d'intermédiaire. Cela fut l'un des points fondamentaux à appliquer, car il a fortement influencé notre conception du projet.

Avant de développer notre nouvelle interface graphique, il a donc fallu réfléchir à la manière de traduire le projet du semestre précédent dans ce format MVC. Cette étape a représenté une partie importante du travail de conception, car notre jeu initial n'avait pas été pensé dès le départ selon cette architecture. Il a donc fallu identifier les responsabilités de chaque classe, séparer la logique métier, le code de l'affichage, et la gestion des interactions, puis organiser ces éléments dans une structure lisible et maintenable.

Vous pouvez retrouver le diagramme complet de cette organisation MVC dans le dossier joint au rapport, car sa taille ne permet pas une intégration lisible dans le document principal.

2. Organisation générale de notre architecture

Pour conserver une architecture claire et exploitable, nous avons choisi de structurer le projet autour de plusieurs ensembles de packages.

Le package "common" regroupe le cœur du projet, c'est-à-dire les classes métier et les objets partagés indépendants de toute interface. On y retrouve par exemple les entités du jeu, les objets, les salles, la génération de donjon, la gestion des langues ou encore les classes utilisées par l'éditeur de niveau. Ce package correspond donc à la base fonctionnelle du projet.

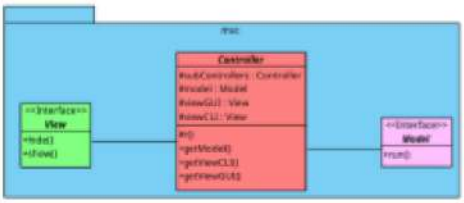
Autour de ce noyau, nous avons organisé le package mvc, qui contient l'ossature applicative du jeu. Ce package reprend les éléments Model, View et Controller, puis l'adapte à notre projet au moyen d'un découpage plus modulaire.

En effet, plutôt que de créer un unique package model, un unique package view et un unique package controller pour tous nos fichiers, nous avons choisi un découpage par domaine fonctionnel. Ainsi, les différentes grandes parties du projet sont séparées dans des sous-packages spécialisés, tels que : entity, map, item, game, menu et le levelEditor. Et c'est

à l'intérieur de chacun de ces domaines que nous retrouvons ensuite une organisation de type MVC avec, selon les besoins, les sous-packages : model, controller, view/base, view/gui, view/cli.
Cette organisation nous a permis de conserver une structure lisible, modulaire et cohérente. Chaque partie du projet reste centrée sur sa responsabilité propre, tout en respectant l'architecture globale imposée.

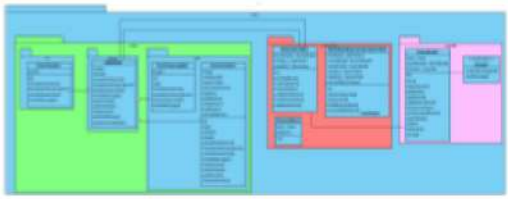
quelque exemple du format MVC de notre projet :

Le core principale, entièrement repris d'un exemple de Mr Skapin



Ce core est connecté à nos différents packages implémentant un modèle MVC. Les classes vont alors implémenter l'une des interfaces (Modèle ou View), ou bien étendre le Controller.

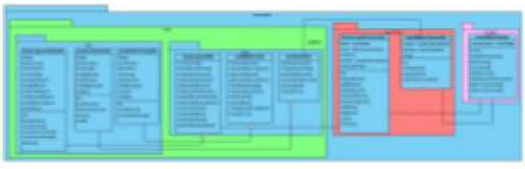
Item : pour la representation des objets et leur connection avec l'environnement du jeu



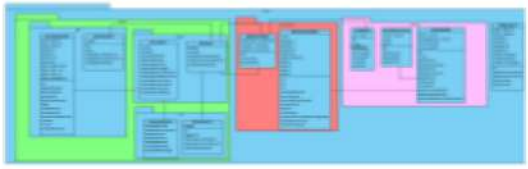
entity : pour la representation des entités comme les joueurs et les enemies et leurs interactions lors de phase de combat



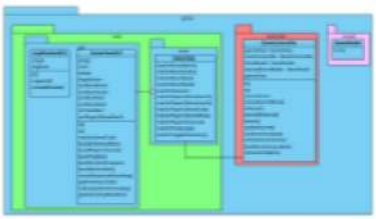
le levelEditor, donc toute le systeme de creation, chargement et sauvegarde de niveau



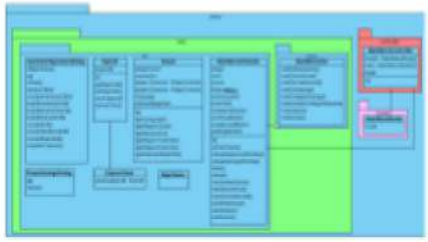
la map, donc ce qui represente l'environnement ou interagisse toute les entité, et ou se passe le jeu.



et enfin Game, package qui fait le lien entre tout les package precedent pour avoir une application fonctionelle



le menu, qui represente.... le menu principale du jeux ainsi que toute les action qui l'entour, comme les parametres ou le systeme de scores....



III - Difficultés rencontrées

Voici ci-dessous une liste des différentes difficultés que nous avons rencontrées lors de la réalisation de ce projet :

- Trouver un accord sur l'architecture MVC à suivre. Plusieurs organisations étaient effectivement possibles :
    - répartir nos fichiers dans trois packages uniques Model, Vue et Controller.
    - mettre en place différents niveaux d'organisation : d'abord par domaine puis par rôle dans l'architecture MVC (comme ce fut le cas pour notre choix final). Une troisième possibilité envisagée suivant ce principe était de créer un premier package par entité, puis dans chaque package les sous packages MVC.

  Nous avons longuement débattu avant de nous orienter vers notre solution, qui se trouve "à mi-chemin" entre les deux autres envisagées.

- L'ajout de la possibilité d'un deuxième joueur ainsi que la gestion des touches de contrôles entre les deux joueurs a été particulièrement complexe à mettre en place. Nous avons en effet décidé de définir des touches par défaut, mais certaines n'étaient pas acceptées : &, é, ", ', etc.

- Bien respecter les principes du modèle MVC, dont notamment aucune communication directe entre la vue et le modèle. Nous avons parfois éprouvé quelques difficultés pour respecter ceci, et une solution a été de décomposer les responsabilités des méthodes afin de mieux identifier les différentes parties du code.

- Un point bloquant que nous avons rencontré fut la gestion de la résolution d'affichage, qui ne fonctionne pas correctement suivant les dimensions de l'écran.

- L'organisation du groupe a été quelque peu compliquée, compte tenu du fait que chaque membre avait des impératifs concernant le stage de fin de licence.

- Certains fichiers comme "RoomController" ou "GameController" ont dépassé les 1000 lignes. Il a alors fallu revoir certains points et être plus consciencieux sur l'organisation des méthodes.

IV - Les principes de la POO dans notre projet

La programmation orientée objet regroupe des principes qui sont au cœur même de notre projet. Elle nous a permis d'organiser le code de manière claire, modulaire et évolutive.

Le jeu repose sur un ensemble de classes spécialisées qui interagissent entre elles, tout en respectant une séparation des responsabilités. Les principaux principes de la POO apparaissent alors à plusieurs niveaux dans notre implémentation.

1. L'Encapsulation

L'encapsulation consiste à regrouper dans une même classe les attributs et les méthodes qui les manipulent, tout en contrôlant leur accès.

Dans notre projet, ce principe est appliqué dans la majorité des classes. Par exemple, la classe Hero contient les attributs propres au héros : son nom, ses points de vie, son arme équipée, son inventaire…

Ces attributs ne sont pas modifiés directement depuis l'extérieur : on passe par des méthodes comme “equipWeapon”, “setCurrentRoom”, “increaseMaxHp” ou encore “healPercentOfMaxHp”.

Autre exemple, la classe Room encapsule toutes les données liées à une salle : son nom, sa description, ses sorties, les objets présents ainsi que les personnages qui s'y trouvent.
L'accès à ces éléments se fait également via des méthodes dédiées comme “addExit”, “addItem”, “removeItem”, “addCharacter” ou “getExit”.

Cette approche évite les modifications incontrôlées et rend le code plus sûr. Chaque classe protège ainsi son propre état interne.

2. L'Abstraction

L'abstraction permet de ne montrer que les éléments essentiels d'un objet, sans exposer tous les détails de son fonctionnement.

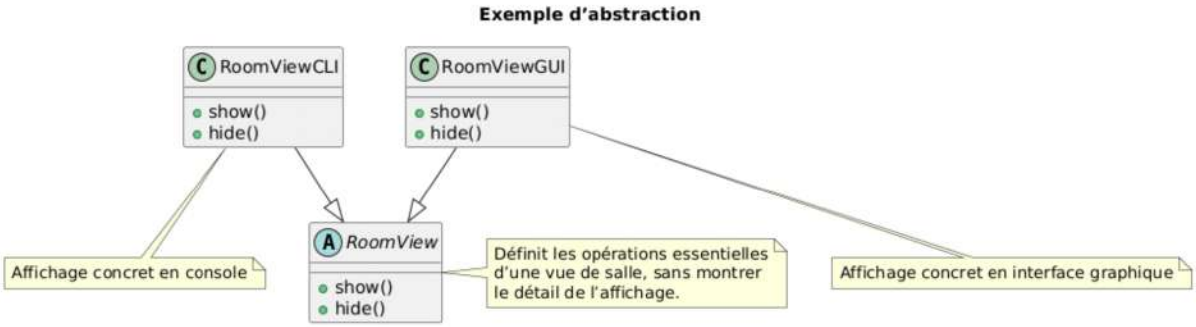
Dans notre architecture, cela apparaît notamment avec les interfaces et les classes abstraites du framework MVC.

Les interfaces Model et View définissent un comportement général commun à plusieurs composants. Par exemple, toutes les vues partagent les méthodes "show" et "hide", même si leur implémentation concrète diffère en fonction des Vues graphiques ou textuelles (Console).

Nous utilisons aussi des classes abstraites comme HeroView, RoomView, ItemView, ExitView ou GameView. Ces classes définissent un cadre commun pour les différentes représentations possibles, sans imposer une seule implémentation.

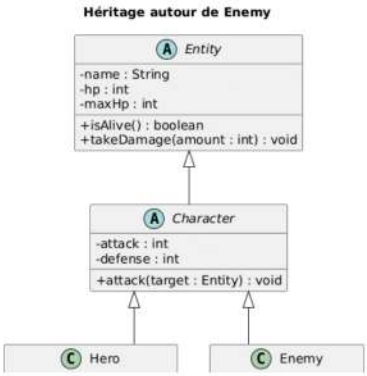
Cela nous permet de manipuler les vues de façon générale dans les contrôleurs, sans dépendre directement des détails d'affichage.

L'abstraction rend donc le projet plus lisible et plus souple, car les contrôleurs travaillent avec des concepts généraux plutôt qu'avec des classes trop spécifiques.



3. L'Héritage

L'héritage permet de créer de nouvelles classes à partir de classes existantes afin de réutiliser du code et de spécialiser certains comportements. Une classe fille hérite donc des attributs et des comportements de sa classe mère, mais elle peut également en avoir d'autres supplémentaires.

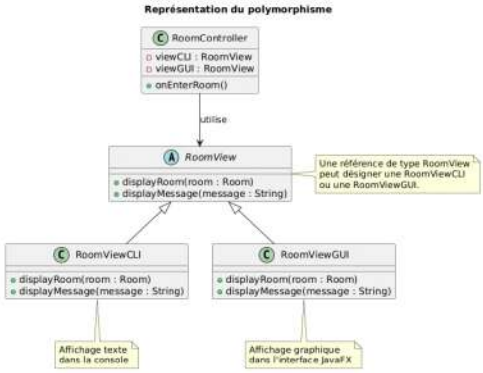


Ce principe est très présent dans notre projet. Par exemple, la classe Hero hérite de la classe Character, tout comme la classe Enemy. Ces deux classes filles sont en effet des personnages et ont donc certains attributs en commun : un nom, des points de vie, un inventaire. La classe mère s'assure de cela. Les deux classes filles ont ensuite d'autres attributs et méthodes qui leurs sont propres.

4. Le Polymorphisme

Le polymorphisme permet d'utiliser des objets de classes différentes à travers un même type général.
Dans notre projet, ce principe est particulièrement visible grâce aux classes abstraites et aux interfaces.

Par exemple, un Contrôleur peut manipuler une variable de type RoomView sans avoir besoin de savoir s'il s'agit exactement d'une RoomViewCLI ou d'une RoomVIewGUI. Les deux respectent la même base mais leur comportement concret diffère.



Le polymorphisme intervient aussi avec les objets du jeu. Une variable de type Item peut en réalité désigner un Weapon, un Scroll, une Key ou un Chest.

Cela simplifie la gestion des inventaires et des interactions, car le programme peut manipuler les objets de façon générique, tout en laissant la bonne méthode s'exécuter selon le type réel de l'objet.

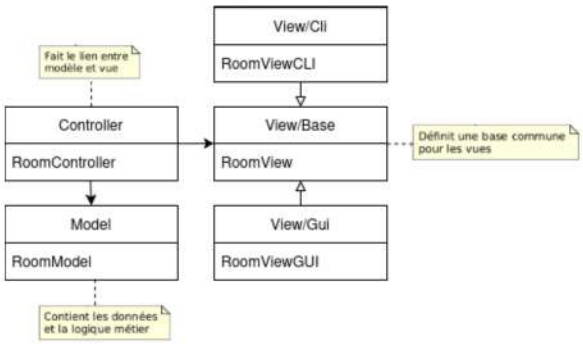
Le même principe existe pour les ennemis dans notre projet : une collection d'ennemis peut contenir à la fois des archers et des berserkers, mais ils seront tous manipulés via le type Enemy.

Cela facilite la gestion des combats et des comportements communs.

5. Une organisation orientée objet adaptée au mvc

Au-delà des quatres grands principes classiques, notre projet montre aussi l'intérêt de la POO dans l'organisation globale du code. L'architecture mvc repose elle-même sur une séparation claire des rôles :

- le modèle contient les données et la logique métier
- la vue gère l'affichage
- le contrôleur fait le lien entre les deux



V - Gestion de projet

1- Choix du projet et organisation générale

Parmi les choix proposés, nous avons choisi celui qui consistait en la reprise du projet du semestre 5, et l'améliorer. N'étant pas dans les mêmes groupes lors de la réalisation de ce projet au semestre dernier, nous nous sommes concertés afin de savoir duquel nous allions repartir. Après avoir étudié les points forts et la complexité de chaque projet nous avons décidé de reprendre celui de Lucas et Léonard, tout en incluant certaines idées venant de celui de Mathis et Tom.

Pour réaliser ce projet, nous avons utilisé l'outil de travail collaboratif Miro, ce qui nous a permis de travailler à plusieurs simultanément. Nous avons pu organiser nos idées, réaliser des diagrammes et des plans tout au long du projet, ce qui a assuré un suivi sérieux du projet.

Nous avons également utilisé le gestionnaire de version Git afin de travailler à plusieurs sur le même code de manière collaborative et efficace.

2- Conception avec l'ajout du MVC

La prochaine étape était de définir l'architecture à suivre. Nous avons passé un certain temps à définir le modèle MVC que nous utiliserons, car plusieurs idées étaient évoquées (détaillées dans la rubrique "Difficultés rencontrées").

Une fois notre modèle choisi, nous avons réalisé ensemble un exemple d'application du MVC afin de s'assurer du bon fonctionnement de ce dernier, puis nous nous sommes répartis les différentes parties du projet, afin de toutes les adapter au format MVC.

Lucas : application
Tom : entity
Mathis : map
Léonard : items

3- Développement et spécialisation des rôles

Après avoir transformé le projet existant en un projet respectant le modèle MVC, nous avons pu nous attaquer au développement des nouvelles fonctionnalités en nous définissant chacun un périmètre de responsabilités :

- Lucas s'est occupé de toute la partie fonctionnelle, c'est-à-dire faire en sorte que les différentes parties s'accordent et forment un ensemble cohérent et fonctionnel.

- Mathis était le garant du respect de la logique MVC, en veillant notamment à la cohérence des abstractions et à ce qu'aucune information ne dépasse de son périmètre.

- Léonard s'est occupé du module de changement de langues, et a notamment commencé l'édition en parallèle de la documentation technique.

- Tom a travaillé sur l'éditeur de niveau, et la gestion du git.

Ce découpage global a permis à chacun d'avancer en parallèle des autres tout en limitant le conflit entre chaque composant. Ill a également servi à répartir efficacement le travail, tout en gardant une cohérence globale dans l'avancement du projet

VI - Conclusion

Nous avons trouvé ce projet intéressant, notamment par la variété des tâches à réaliser.

Dans un premier temps nous avons dû nous solliciter et trouver un accord pour savoir quel projet reprendre et quel direction de développement choisir. Très vite ensuite nous avons échangé et débattu afin de se mettre d'accord sur le modèle MVC à utiliser. Tout cela a demandé un vrai travail de réflexion en groupe, ce qui nous a amené à réfléchir ensemble et à développer davantage notre esprit critique.

Le fait de repartir d'un projet existant pour l'adapter à une architecture MVC nous a permis d'acquérir une nouvelle expérience : ne pas partir d'une feuille blanche mais bien d'un code existant. Il a en effet fallu se familiariser avec ce code et le comprendre avant de le modifier.

La phase de développement qui a suivi, avec l'ajout de la nouvelle interface ainsi que de l'éditeur de niveaux, a été particulièrement complexe.. Il fallait en effet réussir à ajouter de nouvelles fonctionnalités tout en restant cohérent avec notre structure de départ ce qui n'a pas toujours été simple.

Ce projet nous a aussi permis d'appliquer pour la première fois un modèle MVC sur un projet de cette taille, et de gagner en compétences techniques, notamment en JavaFX.

Nous avons aussi pu renforcer nos compétences en organisation et en travail d'équipe via l'utilisation de Miro, la mise en place de développement en pair-programming ainsi que l'utilisation de Git pour la gestion du code (branches, merges, etc.).

Pour conclure, ce projet s'est révélé formateur à différents niveaux et a su nous aider à consolider nos compétences techniques ainsi que notre manière de travailler en groupe.